

Python for CSE 425 — Complete Concept Guide

Every topic explained from scratch, in simple language

PART 1: INTRODUCTION & CORE PHILOSOPHY

1.1 What kind of language is Python?

- **Interpreter-based:** there's no separate "compile" step producing a .exe — an interpreter reads and runs your code line by line. When started directly, it shows `>>>` as a prompt.
- **Dynamically typed:** a variable's type is NOT fixed when created — the same variable can hold an integer now and a string later. Python only checks whether an operation is legal at the moment it RUNS (not before), which is why type errors show up only when that line executes.
- Contrast with **C** (statically typed): in C, a variable is given a fixed type, size, and memory location right at declaration, and it can never change type.
- **No type declarations needed:** you don't write `int x = 7`, just `x = 7`.
- **Readable / concise:** no curly braces `{}` — Python uses **indentation** to mark code blocks, and a colon `:` to say "a block starts here."
- Lists are Python's main all-purpose compound data type (comparable to arrays in other languages, but far more flexible).

1.2 Dynamic typing — the mechanism, precisely

```
x = 1.2      # float
len(x)       # TypeError: object of type 'float' has no len()
x = 7        # int
len(x)       # TypeError: object of type 'int' has no len()
x = "Hello World"
len(x)       # Output: 11
```

`len()` only works on things with a "length" concept (strings, lists, etc.), not raw numbers.

Python doesn't check this until the `len(x)` line actually executes — it re-checks x's CURRENT type at each call, since x can change type at any time. This is dynamic type-checking in action.

PART 2: VARIABLES & FUNCTIONS

2.1 Variables

No type keyword needed — just assign directly:

```
x = 7
y = 20
z = x + y
print('The sum of %d and %d is: %d' % (x, y, z)) # old-style % string formatting
```

The same variable name can later be reassigned to a totally different type:

```
x = 7
x = "Hello World" # now x is a string — completely legal
```

2.2 Defining Functions

Rules:

1. Starts with `def`, then the function name, then `:`
2. No return-type declaration needed.
3. Argument ORDER can be changed at call-time if you name the arguments (keyword arguments).

```
def test_fnc(x):
    evaluate = x * x
    return evaluate

print('Calculated value is %d' % test_fnc(10)) # 100
```

Keyword arguments — order doesn't matter if named:

```
def test_fnc_2(a, b):
    evaluate = a * a + b
    return evaluate

print(test_fnc_2(a=3, b=4)) # 13
print(test_fnc_2(b=4, a=3)) # 13 — same result, order of naming doesn't matter
```

2.3 Worked Example — classify sign of a number

```
def pos_or_neg(x):
    if (x > 0):
        return 'Positive'
    elif (x < 0):
        return 'Negative'
    else:
        return 'Zero'

for i in [10, 12, -1, 0, -5, 10]:
    print(pos_or_neg(i))
```

Output: Positive, Positive, Negative, Zero, Negative, Positive — each element of the list is passed one at a time through the function via the `for` loop.

2.4 Worked Example — circle area

```
import numpy as np
pi = 3.1414
def square(x):
    return x * x
def circle_area(r):
    return pi * r * r
```

Shows how one function (`square`) can be defined and reused, and constants like `pi` can be set once and used across multiple functions.

PART 3: HIGHER ORDER FUNCTIONS (HOF)

3.1 What makes Python functions special

In Python, a function is treated as an **object**, just like a number or a string. This means a function can:

1. Be used as an **argument** to another function.
2. Be **assigned** to a variable.
3. Be **returned** as the value of another function.

Assigning a function to a variable:

```
def test_fnc(x):
    evaluate = x * x
```

return evaluate

```
M = test_fnc      # M now IS test_fnc — no parentheses, so it's not calling it
print(M(10))      # 100 — calling M works exactly like calling test_fnc
```

Important: `M = test_fnc` (no parentheses) assigns the FUNCTION itself to M. `M = test_fnc(10)` would instead assign the RESULT (100) to M — a very common beginner mix-up.

3.2 What is a "Higher Order Function"?

A function that takes another function as one of its arguments (or returns a function). Since functions are objects in Python, you can pass them around freely:

```
def test_fnc(x):
    return x * x
```

```
M = test_fnc
```

```
def higher_func(arg_fun, y):
    final_value = arg_fun(y) * arg_fun(y)
    return final_value
```

```
higher_func(M, 3)  # arg_fun becomes M (= test_fnc), y becomes 3
                  # = test_fnc(3) * test_fnc(3) = 9 * 9 = 81
```

3.3 HOF as a Filter

A common real use of HOFs: writing a function that scans data and keeps only what matches a condition.

```
def TestFilter(data):
    res = []
    for d in data:
        if d >= 50 and d <= 60:
            res.append(d)
    return res
```

```
filtered_data = TestFilter(data)
```

`data` here is loaded from a file with `np.loadtxt('inp.txt')`. The function is "treated as data processing logic" — you feed data through it to get a filtered subset.

3.4 `map`-style pattern (manual implementation)

```
def CSE425_MAP(inp, f):
```

```
    res = []
    for i in inp:
        res.append(f(i))
    return res
```

```
def sq(x):
```

```
    return x * x
```

```
list_example = [5, 1, 3, 7, 4, 8, 9]
```

```
CSE425_MAP(list_example, sq) # [25, 1, 9, 49, 16, 64, 81]
```

This manually recreates what Python's built-in `map()` does: apply a function to every element of a list, one at a time, collecting results into a new list. `f` here is itself a HOF parameter — the caller decides WHICH function gets applied.

PART 4: STRINGS & SLICING

4.1 Basic slicing syntax

`string[start:end]` — includes `start`, EXCLUDES `end`.

```
test_string = 'NorthSouth'
```

```
test_string[0]    # 'N' — first character
```

```
test_string[0:2]  # 'No' — index 0 up to (not including) 2
```

```
sliced_1 = test_string[:5] # 'North' — from start up to (not incl.) index 5
```

```
sliced_2 = test_string[5:] # 'South' — from index 5 to the end
```

```
sliced_1 + sliced_2      # 'NorthSouth' — concatenation with +
```

4.2 Negative indexing

Negative indices count from the END of the string: -1 = last character, -2 = second-last, etc.

```
string_1 = 'THIS IS CSE425 CLASS'
```

```
string_1[-1]    # last character: 'S'
```

```
string_1[-20]    # 20th-from-last character
string_1[-5:-1]  # slice from -5 up to (not including) -1
```

4.3 Iterating over a string

```
for i in 'TEST':
    print(i)
# T
# E
# S
# T
```

A string behaves like a sequence of characters — you can loop over it directly, same as a list.

PART 5: CONTROL STATEMENTS

5.1 Core syntax difference from C

Python uses `:` to mark the start of a block and **indentation** (not `{ }`) to show what's inside it. Indentation is MANDATORY — inconsistent indentation causes an error.

```
if condition < 3:
    print("value_1")
else:
    print("value_2")
```

Equivalent C code needs `{ }` around each block; Python needs neither, just consistent indentation.

5.2 If / elif / else (nested)

```
score_1 = 91
if score_1 >= 93:
    print("Grade A")
elif (score_1 >= 90) and (score_1 < 93):
    print("Grade A-")
elif (score_1 >= 87) and (score_1 < 90):
    print("Grade B+")
else:
    print("Something else")
```

Note: Python uses the WORDS `and` / `or` / `not`, not symbols like `&&` / `||` from C.

5.3 While Loop

```
i = 1
total = 0
while (i <= 10):
    total = total + i
    print(total)
    i = i + 1
```

Runs as long as the condition stays true; `i` must be manually incremented inside the loop, or it never ends (infinite loop).

5.4 For Loop

Unlike C, Python's `for` loop iterates over elements of a sequence (like a list), not a manual counter condition.

```
list_1 = range(1, 10, 2) # 1,3,5,7,9 (start, stop-exclusive, step)
for i in list_1:
    print(i)
```

```
data = [4, 6, 9, 10, 15, 17, 20, 22]
for i in range(1, 8, 2): # i takes values 1,3,5,7
    print(data[i])      # prints data[1], data[3], data[5], data[7]
```

Output: 6, 10, 17, 22 — because `range(1, 8, 2)` generates the INDICES to look up in `data`, not the values themselves.

`range(start, stop, step)` — starts at `start`, stops BEFORE `stop`, jumps by `step` each time.

PART 6: DATA STRUCTURES

6.1 Lists

An ordered, mutable (changeable), comma-separated collection in `[ ]`. Can hold mixed types.

```
list_1 = [1, 2, 3, 4, 5]
list_2 = ['a', 'b', 'c']
list_3 = [1, 2, 3, 4, 5, 'a', 'b', 'c'] # mixed types allowed
list_4 = 4 * [1, 2, 3] # repeats the whole list 4 times → [1,2,3,1,2,3,1,2,3,1,2,3]
```

Common list operations:

```
list_1.append('d') # adds to the END
list_1.insert(3, 'a') # inserts 'a' at index 3, shifting others right
list_1.remove('a') # removes the FIRST occurrence of 'a'
list_1.reverse() # reverses the list IN PLACE
list_1[-1] # last element (negative indexing works on lists too)
```

Copy vs Reference — the single most important list trap:

```
list_1 = [1, 2, 3, 4, 5]
list_2 = list_1 # list_2 is just ANOTHER NAME for the same list in memory
list_1.append(7)
print(list_2) # [1, 2, 3, 4, 5, 7] ← changed too! Same underlying object.
```

Because `list_2 = list_1` copies the REFERENCE (memory address), not the actual data. Both names point to the exact same list object, so a change through either name is visible through both.

Proper copying (breaks the shared-reference link):

```
list_2 = list_1[:] # slicing the whole list creates a NEW list
list_3 = list_1.copy() # .copy() also creates a NEW list
list_1.append(7)
print(list_2) # [1, 2, 3, 4, 5] — unaffected, it's a separate object now
```

Slicing lists (same rule as strings — start included, end excluded):

```
list_1 = [1, 2, 3, 4, 5, 'a', 'b', 'c']
list_1[1:6] # [2, 3, 4, 5, 'a'] — index 1 up to (not incl.) 6
list_1[:5] # [1, 2, 3, 4, 5]
list_1[5:] # ['a', 'b', 'c']
```

Slicing with a step (3rd argument):

```
list_1 = [0,1,2,3,4,5,6,7,8,9]
```

```
even_no = list_1[0:len(list_1):2] # start:stop:step → [0, 2, 4, 6, 8]
```

Nested lists:

```
nested_list = [[1, 2], [3, 4], [5, 6]] # a list containing lists as elements
```

6.2 Tuples

Ordered, like a list, BUT **immutable** — once created, elements cannot be added, removed, or changed.

```
tuple_1 = (1, 2, 5, 7)
tuple_1[0] # 1 — reading is fine
tuple_1.remove(5) # AttributeError: 'tuple' object has no attribute 'remove'
```

Tricky exception: if a tuple CONTAINS a list, that inner list itself can still be modified (the tuple's "slots" are frozen, but a mutable object living in one of those slots is not):

```
tuple_1 = (2, 4, 6, [])
tuple_1[3].append(10)
tuple_1[3].append(12)
print(tuple_1) # (2, 4, 6, [10, 12]) — the list inside changed, tuple structure didn't
```

Tuples for multiple return values:

```
def quotient_remainder_tuple(x, y):
    quot = x // y # floor (integer) division
    rem = x % y # remainder
    return (quot, rem)
```

This is how Python functions return MULTIPLE values at once — packaged as a tuple, then unpacked by the caller: `(q, r) = quotient_remainder_tuple(12, 7)`.

6.3 Strings (as a data structure)

Same indexing/slicing rules as lists (strings are also an ordered sequence type), but strings are IMMUTABLE (you cannot change one character in place — you must build a new string).

```
string_1 = 'THIS IS CSE425 CLASS'
len(string_1) # 20
for i in string_1: # loop over every character
```

```
print(i)
```

6.4 Sets

Unordered collection of UNIQUE elements — duplicates are automatically dropped, and there is no guaranteed order.

```
set_1 = {1, 2, 3}
set_2 = {'a', 'b', 'c', 'b'} # duplicate 'b' — gets collapsed automatically
print(set_2)                 # {'a', 'c', 'b'} — only one 'b', order not guaranteed

set_2.remove('c')            # removes a specific element
set_1.add('c')                # adds a new element
```

Because sets are unordered, you can't reliably index into them like `set_1[0]` — that's what distinguishes sets from lists/tuples.

6.5 Dictionaries

A collection of **key** : **value** pairs — like a set, but each entry carries an associated value. No key can repeat.

```
dict_1 = {'S1': 5, 'S2': 6, 'S3': 7}
dict_1['S4'] = 10    # adding a new key-value pair
```

Iterating a dictionary:

```
for i in dict_1:          # i = each KEY
    print(i)
    print(dict_1[i])      # look up the value using the key

for i, j in dict_1.items(): # i = key, j = value, together
    print('Key {} is {}'.format(i, j))
```

`.items()` gives you BOTH the key and value in each loop step, which is usually cleaner than looking the value up separately.

PART 7: LIST COMPREHENSION

7.1 The basic pattern

A compact, single-line way to build a new list by filtering/transforming an existing one.

Syntax: `[expression for item in iterable if condition]`

```
all_even = [i for i in data if (i % 2) == 0]      # keep even numbers only
divisible_by_4 = [i for i in data if (i % 4 == 0)] # keep multiples of 4
```

Read it almost like English: "give me `i`, for every `i` in `data`, but only if `i % 2 == 0`."

7.2 Chaining / nesting comprehensions

You can nest one comprehension inside another to filter, THEN transform:

```
task = [i**2 for i in [j for j in data if (j % 4 == 0)]]
```

Inner comprehension `[j for j in data if (j % 4 == 0)]` filters first (keep multiples of 4), then the outer comprehension squares each surviving value.

7.3 Matrix generation via nested comprehension

```
matrix = [
    [(a+1)*(i+1) for a in range(3)]
    for i in range(3)
]
# [[1, 2, 3], [2, 4, 6], [3, 6, 9]]
```

The OUTER loop (`for i in range(3)`) picks each ROW; the INNER loop (`for a in range(3)`) fills in each row's values. This is exactly how you'd build a 2D grid/matrix using pure list comprehension, without explicit nested `for` loops.

7.4 Quicksort using list comprehension

```
def quicksort_list(input_array):
    if len(input_array) <= 1:
        return input_array
    pivot_element = input_array[len(input_array) // 2]
    left = [x for x in input_array if x < pivot_element]
    middle = [x for x in input_array if x == pivot_element]
    right = [x for x in input_array if x > pivot_element]
    return quicksort_list(left) + middle + quicksort_list(right)
```

How it works (divide and conquer):

1. Pick a **pivot** (here, the middle element).
 2. Split the array into three lists: **left** (smaller than pivot), **middle** (equal to pivot), **right** (bigger than pivot) — each built with a one-line list comprehension.
 3. Recursively sort **left** and **right** the same way.
 4. Glue everything back together: **sorted(left) + middle + sorted(right)**.
 5. The recursion's BASE CASE is **len(input_array) <= 1** — a list of 0 or 1 elements is already sorted, so it's returned as-is (this is what stops the recursion, exactly like a base fact in Prolog or a base case in Lisp factorial).
-

PART 8: OBJECT-ORIENTED PROGRAMMING (OOP)

8.1 Objects

An **object** bundles together DATA (attributes) and BEHAVIOR (methods/functions) into one unit.

8.2 Class Definition

A **class** is the blueprint/template used to create objects.

```
class basicMath:
    def subtraction(self, a, b):
        return a - b
    def product(self, a, b):
        return a * b
    def addition(self, a, b):
        return a + b
    def division(self, a, b):
        return a / b

m = basicMath()      # creating an INSTANCE (object) of the class
x1 = m.subtraction(8, 6) # 2
x4 = m.division(8, 6)   # 1.33...
```

self is the first parameter of every method — it refers to "this particular object" (the instance calling the method), letting the method access that specific object's own data.

8.3 Inheritance — Parent vs Child

A `Child` class can inherit everything from a `Parent` class:

```
class Parent():
    def test_fnc1(self):
        print('test-one print in Parent')
    def test_fnc2(self):
        print('test-two in Parent')

class Child(Parent):
    pass # inherits everything from Parent, adds nothing new

value2 = Child()
value2.test_fnc1() # 'test-one print in Parent' — inherited method works!
```

`class Child(Parent):` means `Child` automatically gets ALL of `Parent`'s methods for free, without rewriting them. `pass` is a placeholder meaning "no additional code here."

8.4 The `__init__()` Constructor

`__init__()` is a special built-in method Python calls AUTOMATICALLY whenever a new object is created — it's used to set up the object's initial data (attributes).

```
class cse425student:
    def __init__(self, name, id, sec, grade):
        self.name = name
        self.id = id
        self.sec = sec
        self.grade = grade
```

```
s1 = cse425student("karim", 23, 1, "A")
print(s1.name) # karim
```

`self.name = name` stores the passed-in value onto THIS specific object, so each object created from the class can hold its own independent data (e.g., a second student object would have a totally different `.name`).

PART 9: CONCURRENCY & PARALLELISM

9.1 Multiprocessor Architecture

Modern computers have multiple processor cores capable of running tasks at the same time. If software isn't specifically written to use multiple processors, that extra hardware capability goes unused.

9.2 Concurrency

Multiple activities happening "simultaneously" without interfering with each other. Examples: several computers on a network, or several apps running on one machine, or a web browser simultaneously sending/receiving data, rendering the page, AND responding to clicks.

Levels of concurrency (from smallest to largest scope):

1. **Machine instruction level** — individual CPU instructions run concurrently.
2. **Statement level** — multiple program statements execute at once.
3. **Subprogram level** — multiple functions/subroutines run concurrently.
4. **Program level** — multiple whole programs run at once.

(Statement-level and Subprogram-level concurrency are the two levels most relevant to programming LANGUAGE design, as opposed to OS/hardware-level concerns.)

Example — sequential vs multiprocessing:

```
import time
def mt():
    for i in range(5):
        print(i)
        time.sleep(1)
mt()      # runs one iteration at a time, 1 second apart (SEQUENTIAL, not concurrent)
```

```
import time
from multiprocessing import Pool

def some_func(i):
    print(i)
    time.sleep(1)

def main():
    p = Pool(processes=5)      # spin up 5 worker processes
    result = p.map(some_func, range(5)) # distribute the 5 calls across the 5 workers

if __name__ == '__main__':
    main()
```

`Pool(processes=5)` creates 5 separate worker processes; `p.map(some_func, range(5))` hands out one call of `some_func` to EACH worker, so (unlike the sequential version) all 5 can run roughly AT THE SAME TIME instead of one after another.

9.3 Parallelism (contrast with Concurrency)

Concurrency = structuring a program to HANDLE multiple tasks (may or may not literally run at the same physical instant — could be time-sliced on one core). Parallelism = tasks physically executing at the exact same time on separate processing units (multiple cores/CPU's). Parallelism requires actual concurrent-capable hardware; concurrency is more about program STRUCTURE/design.

PART 10: NUMERICAL SIMULATION — LINEAR ALGEBRA WITH NUMPY

10.1 Vectors

A vector's *i*-th element is accessed the same way as a list, indexing starting from 0.

```
import numpy as np
x = np.array([2, 4, 35])
x[0]      # 2
```

```
vector_1 = 10 * np.ones(4, dtype=int)  # [10, 10, 10, 10] — a vector of four 10s
```

`np.ones(4)` creates a vector of 4 ones; multiplying by 10 scales every element to 10. `dtype=int` forces integer type (otherwise NumPy defaults to floats).

10.2 Vector Operations

Vector + Vector (element-by-element addition):

```
vector_1 = 10 * np.ones(4, dtype=int)  # [10,10,10,10]
vector_2 = 5 * np.ones(4, dtype=int)   # [5,5,5,5]
c = vector_1 + vector_2                 # [15,15,15,15] — added position-by-position
```

Vector + Scalar (broadcasting):

```
vector_3 = 4 * np.ones(5, dtype=int)   # [4,4,4,4,4]
```

`vector_4 = vector_3 + 1` `# [5,5,5,5,5]` — the scalar 1 is added to EVERY element

This "stretching" of a single scalar value across every element of a vector is called **broadcasting** — NumPy automatically applies the scalar operation to each element without you writing a loop.

QUICK REFERENCE TABLE — SYNTAX CHEAT SHEET

Concept	Python syntax	Notes
Variable	<code>x = 7</code>	No type keyword
Function	<code>def f(x): ... return y</code>	No return type declared
If block	<code>if cond:</code>	Colon + indentation, no <code>{ }</code>
Loop over list	<code>for i in lst:</code>	Iterates elements, not indices
While	<code>while cond:</code>	Must manually update loop variable
List	<code>[1,2,3]</code>	Ordered, mutable, mixed types OK
Tuple	<code>(1,2,3)</code>	Ordered, immutable
Set	<code>{1,2,3}</code>	Unordered, unique elements only
Dict	<code>{'k':'v'}</code>	Key-value pairs, unique keys
Slicing	<code>lst[start:end:step]</code>	start incl., end excl.
List comprehension	<code>[expr for i in lst if cond]</code>	Compact filter+transform
Class	<code>class Name: def __init__(self,...): ...</code>	<code>self</code> = current object
Inheritance	<code>class Child(Parent):</code>	Gets all Parent's methods
Copy list safely	<code>lst[:]</code> or <code>lst.copy()</code>	Plain = only copies the reference

